

# Volvo Cars — Product Detail Pages Architecture Overhaul

---

**Role:** Senior Product Engineer (Tech Lead)

**Company:** Volvo Cars

---

## Opener / Hook

"I inherited Volvo Cars' most-trafficked commercial pages and led a full architectural rebuild that moved every Core Web Vital from Poor to Good for 200K+ daily visitors across multiple markets."

---

## Situation (S)

- Product detail pages (core commercial surface — 200,000+ daily visitors across multiple markets) were built with **client-side data fetching** via hooks/providers and lazy-loaded components.
  - **SSR was already in use**, but data fetching was not centralized — components fetched data after hydration, so content was visibly loading as users scrolled.
  - **CDN caching was in place** but not utilized effectively — none of the client-fetched data was cached.
  - **Core Web Vitals** were in the Poor or high Needs Improvement range across LCP, INP, and CLS.
  - **SEO** was poor: lazy loading meant only small fractions of each page were crawlable.
  - Developer experience was poor — data flow was hard to follow, causing frustration on the team.
- 

## Task (T)

- A planned product redesign (new components built from scratch) created an opportunity to rethink the architecture at the same time.
  - My objective: propose and lead a migration to **server-side data fetching**, centralized via **Next.js app router and server components**, to improve performance, SEO, and DX simultaneously.
  - Scope: SSR data fetching migration, CDN caching optimization, and performance monitoring setup.
  - Required alignment with **product, UX, and backend teams** across multiple markets.
  - **Timeline:** ~6 months from kickoff to first launch (architecture, design, and component build happening in parallel). Incremental improvements have continued at smaller scale since.
- 

## Action (A)

- **Proposed and designed** the migration to Next.js app router + server components, timed to coincide with the redesign to avoid double-touching every component.
- **Migrated data fetching to the server:** data available immediately at render time, eliminating post-hydration loading.
- **Optimized CDN caching** so server-fetched data was cached, reducing TTFB at scale.

- **Removed unnecessary lazy loading and client-side data fetching** for above-the-fold content — full page content now available on first load (SEO + accessibility benefit).
  - **Set up performance monitoring** with Elastic and Grafana to track regressions during rollout and validate improvements.
  - **Led the team** (4–5 engineers): set engineering standards, drove code reviews, stayed hands-on with core migration work.
  - **Got backend buy-in** to change how data was served — required cross-team alignment, not just a technical change.
  - **Redesigned CMS content types** to be more scalable alongside the architectural changes — improved the editor experience and made it easier to extend content across markets without engineering involvement or duplicate work.
- 

## Result (R)

- **All Core Web Vitals moved from Poor/Needs Improvement → Good:**
    - LCP: **~4.8s -> ~1.5s**
    - INP: **~170ms** (Before value not available as it was measured as FID rather than INP)
    - CLS: **~0**
  - **SEO improved** — full page now crawlable; search rankings improved across markets.
  - **Accessibility improved** as a side effect of removing lazy loading and client-side rendering.
  - **TTFB reduced** — CDN caching made the pages reliably fast at 200K+ daily visitors.
  - **Developer experience improved** — clear, traceable data flow via server components; CMS content types were redesigned to be more scalable, making it easier for editors to manage and extend content across markets.
- 

## Reflection (R)

- **Observability made this possible.** We watched LCP, INP, TTFB, and error rates in Elastic and Grafana through the rollout — without that, we couldn't have validated improvements confidently or caught regressions early. The signals you choose to watch and how you act on them is the whole game in a migration like this.
- **Performance work is often organizational, not just technical.** Getting backend teams to change how data was served required as much stakeholder alignment as engineering.
- **Timing the migration with the redesign** was the key unlock — it let us rethink the architecture without paying the cost of double-touching every component.
- Legacy architecture can accumulate in subtle ways (CDN not caching the right data, SSR without centralized fetching) — full impact only becomes clear when you fix all the pieces together.